

Signing in from the ERP

The Timetable module has **no login of its own**. Users arrive from the ERP menu carrying a short-lived, RS256-signed token; the Timetable verifies it, provisions the user and their school on the spot, and issues its own session. This is everything the ERP team needs to mint that token.

How it works

Four steps. The ERP does step 1; everything after it is ours.

- 1

ERP

Mint a signed token

When a user clicks the Timetable item in the ERP menu, sign a short-lived RS256 JWT describing who they are, what role they hold, and which school they are opening.
- 2

ERP

Redirect to the callback

Send the browser to our callback URL with the token in the query string. No POST, no form, no shared cookie — a plain redirect.
- 3

TIMETABLE

Verify, provision, mint a session

We check the signature and expiry, burn the token's nonce so it cannot be reused, create or refresh the school and the user, and apply the role your token names.
- 4

TIMETABLE

Redirect into the app

The browser lands in the Timetable web app carrying our own 8-hour session token. The ERP's token is discarded and never used again.

The endpoint

One public endpoint. It is a browser redirect target, not an AJAX call.

```
GET {TIMETABLE_API_BASE}/api/sso/callback?token={JWT}
```

PARAMETER		DESCRIPTION
<code>token</code>	REQUIRED	The compact RS256 JWT described below. JWTs are base64url so they are URL-safe as-is, but URL-encode anyway if your HTTP client does not.

Note the `/api` prefix — every route on the service carries it. In our development stack the base is `http://localhost:3001`; for your environments we will give you the real hostname.

Open it in the same tab

The response is a 302 into the Timetable web app, which then holds an 8-hour session. Opening it in a new tab works equally well — what matters is that a *browser* follows the redirect, since the session lands in the URL fragment.

Token header

RS256 only. HS256 and every other algorithm is rejected — we hold your public key, never a shared secret, so a symmetric algorithm would mean giving both sides the power to mint tokens.

JOSE HEADER

```
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "edunext-prod-2026" // optional, but recommended – see below
}
```

FIELD		NOTES
<code>alg</code>	REQUIRED	Must be RS256 . Anything else fails verification.
<code>kid</code>	RECOMMENDED	Key id. If present, it selects your registered public key. An unrecognised <code>kid</code> is rejected outright — it never falls back to a default key, because falling back is how one ERP installation would end up trusted to sign for another's schools. Omit it and we use the single configured key.

Token claims

The payload. Five required claims, the rest optional.

CLAIM	TYPE		DESCRIPTION
<code>erpUserId</code>	string	REQUIRED	Your stable identifier for this user. It is the key we match on, so it must not change for a given person. Max 50 characters.
<code>name</code>	string	REQUIRED	Display name. Refreshed on every login, so a rename in the ERP propagates without anyone retyping it. Max 100 characters.
<code>email</code>	string	REQUIRED	Used for display and notifications. Max 120 characters.
<code>erpRole</code>	string	REQUIRED	Your role name for this user in this school. Mapped to a Timetable role — see Roles. An unmapped value is refused.
<code>school</code>	object	REQUIRED	The school this session opens in. See School identity.
<code>jti</code>	string	REQUIRED	A nonce, unique to every token you mint. We store it and refuse a second presentation. A UUID is ideal.
<code>exp</code>	number	REQUIRED	Standard JWT expiry, in seconds since the epoch. Keep it short — see Rules we enforce.
<code>schools</code>	array	OPTIONAL	Every school this user may work in. Drives the in-app school switcher. See Multi-school.
<code>trust</code>	object	OPTIONAL	<code>{ code, name }</code> of the trust or management body these schools belong to.
<code>teacherId</code>	number	OPTIONAL	Links this login to a teacher record. Read Teacher linking before sending it.
<code>iss</code>	string	OPTIONAL	Your issuer identifier. A fallback way to select your public key when no <code>kid</code> is present.
<code>schoolId</code>	number	LEGACY	A numeric id, honoured only for deployments predating the <code>school</code> claim. Do not use it for a new integration: a number carries no name, so it can only find a school that already exists and can never create one.

School identity

The ERP owns school identity. We never hardcode or invent a school name — it arrives on the token, and an unknown school is created on the spot, complete with its permission registry and role mappings. Names are refreshed on every login.

THE `SCHOOL` CLAIM

```
{
  "code":      "DPS-N0IDA-01",      // required – the stable key between our systems
  "name":      "Demo Public School", // required
  "shortName": "DPS Noida",         // optional
  "logoUrl":   "https://.../logo.png", // optional
  "timezone":  "Asia/Kolkata",       // optional, defaults to Asia/Kolkata
  "address":   "Sector 62, Noida"    // optional
}
```

`code` must be stable and unique, forever

It is the only identifier the two systems share. It is what the master-data sync matches on, what decides which database a school's rows live in, and what school switching is authorised against. Changing a school's code after go-live orphans its data. Pick something that will never be edited for cosmetic reasons — not the display name.

Maximum lengths: `code` 40, `name` 120, `shortName` 40, `timezone` 40, `logoUrl` and `address` 255.

Roles

Send *your* role name in `erpRole`. We map it to a Timetable role, per school. Every newly provisioned school starts with these four mappings:

ERPROLE YOU SEND	TIMETABLE ROLE	WHAT THEY GET
ADMIN	Super Admin	Everything, including master data, generation and the role editor.
PRINCIPAL	Principal	Full visibility across the school; publishing and reports.
TEACHER	Teacher	Scoped views — their own timetable and their linked class-sections only.
FRONT_OFFICE	Front Office	Substitute management and day-to-day operations.

An unmapped role is refused, not defaulted

If you send an `erpRole` with no mapping in that school, sign-in fails with *"No timetable role mapped for ERP role 'X'"*. This is deliberate — silently granting a default role to an unrecognised one is how someone ends up with authority nobody chose to give them. Send us the exact list of role strings you will use and we will map them, or a school Admin can map them in-app on the Roles & Access page.

The role is re-applied from your token on every login, so a promotion in the ERP takes effect immediately — *unless* a school Admin has deliberately overridden that user's role in the Timetable, in which case their override stands.

Teacher linking

A user with the Teacher role sees only their own grid and their linked class-sections. That scoping needs the login connected to a teacher record, which is what `teacherId` does.

Open item — please read before implementing

`teacherId` is currently **our internal numeric id**, which your ERP has no natural way of knowing. That is a gap on our side, not yours.

For now: **omit** `teacherId`. Teacher logins will work and see the app, but their personal timetable scope will be empty until the link exists.

The fix we intend is to match on **employee code** instead — the same identifier the master-data sync already uses for staff — so the ERP sends something it actually owns. Tell us the field name you use for employee code and we will wire it up.

Multi-school and trusts

A user who works across several schools — a trust administrator, a teacher shared between branches — gets a `schools` array listing every school they may open. The `school` claim decides which one the session *starts* in.

TRUST ADMINISTRATOR TOKEN, ABBREVIATED

```
{
  "erpUserId": "EDX-10422",
  "erpRole": "ADMIN",
  "trust": { "code": "EDUNEXT-TRUST", "name": "Edunext Education Trust" },
  "school": { "code": "DPS-NOIDA-01", "name": "Demo Public School" },
  "schools": [
    { "code": "DPS-NOIDA-01", "name": "Demo Public School" },
    { "code": "DPS-GZB-02", "name": "Second Branch" }
  ],
  // ... erpUserId, name, email, jti, exp as usual
}
```

Every school in `schools` is created or refreshed at login, and the in-app switcher offers exactly that list. **Switching is authorised by the signed token, never by a permission** — the server will not switch to a school that was not on it. A user's role is resolved per school, so the same person can be an Admin in one and a Teacher in another.

Single-school users can omit `schools` and `trust` entirely.

Rules we enforce

RULE	WHAT HAPPENS IF BROKEN
RS256 signature , verified against your registered public key	Rejected. Wrong algorithm, wrong key, or a tampered payload all fail here.
Not expired (<code>exp</code>)	Rejected.
<code>jti</code> not seen before	Rejected — <i>"SSO token replayed"</i> . Each token opens exactly one session.
A registered <code>kid</code> , when one is present	Rejected. No fallback to a default key.
A mapped <code>erpRole</code>	Rejected, naming the unmapped role.
An active user	Rejected if the user was deactivated in the Timetable.
An active school	Rejected if the school is suspended.

Keep `exp` at 60 seconds

The nonce that makes a token single-use is remembered for **120 seconds**. A token that outlives that window is, in principle, replayable again afterwards. A sign-in token only has to survive one redirect, so 60 seconds is generous — our own test issuer uses exactly that. Do not issue minutes-long or hour-long SSO tokens.

The token is **consumed once and discarded**. We do not store it and it cannot be presented back to your APIs — which is why the master-data sync authenticates separately, with OAuth2 client credentials.

Success and failure

On success

A `302` to the Timetable web app, carrying our own session token in the URL fragment:

```
Location: https://timetable.example.com/sso#token=<session JWT, valid 8 hours>
```

The fragment is deliberate: it never reaches a server log or a `Referer` header. The web app reads it, stores it, and replaces the URL. Nothing further is required of the ERP.

On failure

```
Location: https://timetable.example.com/sso-error
```

The user sees a plain "return to the ERP and open Timetable again" page. **The browser is told nothing about why** — the specific reason is written to our server log instead, so a bad token and an operational fault are distinguishable to an operator but not to an attacker. If sign-in fails during integration, ask us for the log line; it names the exact cause.

Worked example

COMPLETE PAYLOAD — A SCHOOL ADMINISTRATOR

```
{
  "erpUserId": "EDX-10422",
  "name": "Aditi Verma",
  "email": "aditi.verma@dpsnoida.edu.in",
  "erpRole": "ADMIN",
  "school": {
    "code": "DPS-N0IDA-01",
    "name": "Demo Public School",
    "shortName": "DPS Noida",
    "timezone": "Asia/Kolkata"
  },
  "jti": "7c1f9e64-3b2a-4d51-9f7e-0a2b6c8d1e34",
  "exp": 1788249660
}
```

PHP

```
use Firebase\JWT\JWT;

$payload = [
  'erpUserId' => $user->code,
  'name'      => $user->full_name,
  'email'     => $user->email,
  'erpRole'   => $user->role_code,           // ADMIN | PRINCIPAL | TEACHER | FRONT_OFFICE
  'school'    => [
    'code' => $school->code,
    'name' => $school->name,
  ],
  'jti' => (string) Str::uuid(),
  'exp' => time() + 60,
];

$token = JWT::encode($payload, $privateKeyPem, 'RS256', 'edunext-prod-2026');

return redirect('https://timetable.example.com/api/sso/callback?token=' . $token);
```

Node

```
const jwt = require("jsonwebtoken");
const { randomUUID } = require("node:crypto");

const token = jwt.sign(
  {
    erpUserId: user.code,
    name:      user.fullName,
    email:     user.email,
    erpRole:   user.roleCode,
    school:    { code: school.code, name: school.name },
  },
  privateKeyPem,
  { algorithm: "RS256", expiresIn: "60s", jwtid: randomUUID(), keyid: "edunext-prod-2026" },
);

res.redirect(`https://timetable.example.com/api/sso/callback?token=${token}`);
```

Both libraries set `iat` automatically. `expiresIn` / `exp` and `jwtid` / `jti` are the same claims under the libraries' own names.

What each side needs

From the ERP team

- **RSA public key**, PEM format (2048-bit or better). We never need the private key.
- **The `kid`** you will stamp on tokens, and your `iss` value if you set one.
- **The exact `erpRole` strings** you will send, so we can map any beyond the four defaults.
- **The school codes** you will use, and confirmation they are stable.
- Your **employee-code field name**, if you want teacher linking (see Teacher linking).

From us

- The **callback URL** for each environment.
- Confirmation your key is registered and which `kid` it answers to.
- Role mappings applied for any non-default roles.
- Server-side log access during integration, so a failed sign-in has a reason rather than a shrug.

Key rotation is supported: several keys can be registered at once, each under its own `kid`, so you can roll over without a flag day.

Testing the integration

Work through it in this order — each step isolates a different thing, so a failure tells you where the problem is.

1. **Decode your own token** at any JWT decoder and check the claims match the table above. Most integration failures are a missing `jti` or a misspelt claim name.
2. **Confirm the signature** verifies against the public key you sent us, using the key file itself rather than the one in your config.
3. **Open the callback in a browser.** Landing on the Timetable dashboard means the whole chain works. Landing on `/sso-error` means ask us for the log line.
4. **Try a second click within the same minute.** A fresh `jti` each time must succeed; reusing one token twice must fail. If reuse succeeds, your `jti` is not unique.
5. **Try each role** you intend to send, and confirm the user lands with the expected menu.

A test issuer exists in our development stack

Our non-production environment can mint tokens with a built-in key, which lets us reproduce a scenario without involving your systems. It is disabled in production and its key does not exist there. Useful when we need to isolate whether a problem is in the token or in us.

Edunext ERP → EduTimetable · SSO integration reference. Questions on any claim, rule or failure mode go back to the Timetable team — particularly the teacher-linking item, which needs a decision from both sides.